



5.2. Data Grouping and Transforming

Previous Section:

 [5.1. Data Grouping and Aggregation](#)

Contents:

[1. Creating Derived Columns Using Group Information](#)

[Example 1. Creating a Final GPA](#)

[Example 2. Creating a Scholarship Column](#)

[Example 3. Creating Derived Columns with `transform\(\)`](#)

[2. Imputing Missing Values with Group Statistics](#)

[3. Other Common Transformations](#)

[Finding Each Student's Contribution to Their Group](#)

[Group-wise Standardization \(Z-Score\)](#)

[Summary](#)

[References](#)

While aggregation summarizes data into a smaller table, **transformation** returns a value for **every row** in the original dataframe based on information from its group. This makes grouping useful not only for reporting but also for feature engineering, missing value imputation, normalization, and many other preprocessing tasks.

The most commonly used method is `transform()`, which performs a group-wise calculation while preserving the original dataframe size.

Throughout this section, we will continue using our synthetic student dataset.

```
import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)
```

```

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))

```

1. Creating Derived Columns Using Group Information

A **derived column** is a new column created from one or more existing columns using mathematical operations, logical conditions, or group statistics.

Example 1. Creating a Final GPA

Suppose the university introduces a second assessment score (`GPA2`). We first rename the original GPA column to `GPA1`, generate a new GPA, and then compute the final GPA.

```

# Rename original GPA
df.rename(columns={"GPA": "GPA1"}, inplace=True)

# Generate another GPA
df["GPA2"] = np.round(np.random.uniform(2.0, 4.0, len(df)), 2)
# Final GPA
df["Final_GPA"] = (df["GPA1"] + df["GPA2"]) / 2

print(df.head())

```

Example 2. Creating a Scholarship Column

Suppose scholarships are awarded according to the following rules:

- **Web Engineering Scholarship:**
 - Major is **SW** or **CSC**
 - Final GPA ≥ 3.2
- **Data Science Scholarship:**
 - Major is **AI** or **IT**
 - Final GPA ≥ 3.2

Otherwise, students receive **No Scholarship**.

```
def scholarship(row):  
    if row["Major"] in ["SW", "CSC"] and row["Final_GPA"] >= 3.2:  
        return "Web Engineering"  
    elif row["Major"] in ["AI", "IT"] and row["Final_GPA"] >= 3.2:  
        return "Data Science"  
    else:  
        return "None"  
  
df["Scholarship"] = df.apply(scholarship, axis=1)  
print(df[["Name", "Major", "Final_GPA", "Scholarship"]].head())
```

The previous examples create new columns using existing values in each row. However, many real-world features depend on **how a row compares with other rows in the same group**, which is where `transform()` becomes useful.

Example 3. Creating Derived Columns with `transform()`

Suppose we want to know whether a student's GPA is **above or below the average GPA of their major**.

First, compute the average GPA for each major and assign it back to every student in that major.

```
df["Major_Avg_GPA"] = df.groupby("Major")["Final_GPA"].transform  
("mean")
```

Then create a new column indicating whether each student is above the group average.

```
df["Above_Average"] = (df["Final_GPA"] > df["Major_Avg_GPA"])
print(df.head())
```

Unlike `agg()`, the `transform()` method returns one value **for every original row**, making it ideal for creating derived features.

2. Imputing Missing Values with Group Statistics

Previously, we replaced missing values using the **overall mean** of the dataset.

However, this may not always produce the best estimate. Students from different majors may have different GPA distributions, so replacing missing GPAs using the average GPA of the corresponding major often produces more realistic values.

- Suppose several GPA values are missing.

```
df.loc[[2, 6, 11], "Final_GPA"] = np.nan
```

- Imputation using the global mean:

```
df["Final_GPA"] = df["Final_GPA"].fillna(df["Final_GPA"].mean())
```

- Instead of using the global mean, we can fill missing values using each major's average GPA.

```
df["Final_GPA"] = df["Final_GPA"].fillna(
    df.groupby("Major")["Final_GPA"]
    .transform("mean"))
```

Since `transform()` returns one value for every row, each missing GPA is replaced by the average GPA of students in the same major.

This approach often preserves the natural characteristics of each group better than using a single global average.

3. Other Common Transformations

The `transform()` method is widely used for many other data preprocessing tasks.

Finding Each Student's Contribution to Their Group

The following calculates what percentage of the total GPA for each major is contributed by an individual student.

```
df["Contribution"] = (df["Final_GPA"] / df.groupby("Major")["Final_GPA"]  
                        .transform("sum"))
```

Since they are percentages. You can use `np.round(df_new1["Contribution"] * 100, 2)` to convert them to meaningful percentage values

Optionally, you can filter out any major group

Group-wise Standardization (Z-Score)

Sometimes we want to normalize data **within each group** rather than across the entire dataset.

```
df["Z_Score"] = (df.groupby("Major")["Final_GPA"]  
                  .transform(lambda x: (x - x.mean()  
                  ) / x.std()))
```

Students from different majors are now compared relative to other students in the same major rather than across the entire university.



`transform()` VS `agg()`

Although both methods are used after `groupby()`, they serve different purposes.

<code>agg()</code>	<code>transform()</code>
Produces one summary value per group	Produces one value for every original row
Reduces the size of the dataframe	Preserves the original dataframe size
Used for reporting and summaries	Used for feature engineering and preprocessing
Returns aggregated results	Returns transformed values aligned with the original index

- For example, `df.groupby("Major")["Final_GPA"].mean()` returns one average GPA for each major.
- Whereas, `df.groupby("Major")["Final_GPA"].transform("mean")` returns the average GPA of each student's major, repeated for every student in the dataframe.

Summary

While `groupby()` and `agg()` are primarily used to summarize data, the `transform()` method enables group-based calculations while preserving the original dataframe structure.

This makes it especially useful for creating derived features, imputing missing values using localized group statistics, computing relative contributions, and performing group-wise normalization.

Together, aggregation and transformation provide a powerful toolkit for both exploratory data analysis and data preprocessing.

To practice further, complete Lab 05 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.core.groupby.DataFrameGroupBy.transform.html>

<https://www.geeksforgeeks.org/pandas/python-pandas-dataframe-groupby/>

Next Section:

6.1. Data Normalization